LECTURA PREVIA
EDICIÓN 2026

METSI

N29
FCE · UBA

N29

Una liberación exige evidencia común de versión, exposición, recuperación y autoridad.

METODOLOGÍA DEL ESTUDIO DE SISTEMAS DE INFORMACIÓN

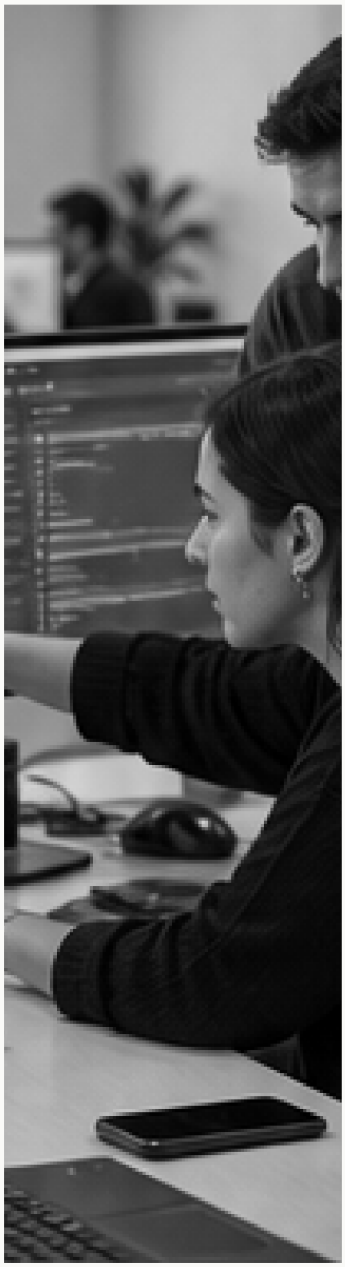
Cambiar sin dejar de atender

Entrega continua, exposición, recuperación y control del cambio

Contenido

Cambiar el sistema sin perder el control del servicio

Ruta de preparación: leé las entradas NÚCLEO y elaborá el producto de apertura. EXT. amplía el recorrido; glosario y referencias sirven de consulta. Las precisiones de la apertura distinguen extensiones dentro de un movimiento.



Una lectura previa para llegar al encuentro con preguntas, evidencia y una decisión revisable.

- Referentes SIN NUM. p. 3

01 Pregunta profesional NÚCLEO p. 4

02 Volvió el programa anterior. Los pagos seguían duplicados NÚCLEO p. 6

03 Hotel Horizonte: el botón de volver no desbloqueaba las habitaciones NÚCLEO p. 7

04 Tesis NÚCLEO p. 9

05 De la calidad esperada a la decisión de liberar EXT. p. 11

06 Las ideas que sostienen el recorrido EXT. p. 11

07 Movimiento 1 · Saber exactamente qué estamos cambiando NÚCLEO p. 12

08 DevOps: construir y operar sin pasarse el problema EXT. p. 15

09 Movimiento 2 · Decidir quién recibe el cambio y bajo qué condiciones NÚCLEO p. 16

10 Movimiento 3 · Volver, corregir y reparar son acciones distintas NÚCLEO p. 19

11 Instrumento HH-29: el plan de cambio y recuperación NÚCLEO p. 22

12 Ensayar una recuperación antes de necesitarla NÚCLEO p. 27

13 Cerrar la liberación sin retirar el apoyo demasiado pronto NÚCLEO p. 27

14 Transferencia: una regla de liquidación bancaria EXT. p. 28

15 Contraejemplo: automatizar todos los pasos equivocados EXT. p. 29

16 Errores frecuentes EXT. p. 30

17 Consecuencias profesionales EXT. p. 31

18 Límites y tensiones EXT. p. 31

19 Una lectura situada desde América Latina EXT. p. 32

20 De N29 a N30 EXT. p. 33

21 Síntesis NÚCLEO p. 33

22 Cinco píldoras para recordar EXT. p. 34

- Del concepto al producto SIN NUM. p. 35

23 Glosario esencial EXT. p. 36

24 Preguntas de preparación NÚCLEO p. 37

- Referencias base SIN NUM. p. 38

Nota. SIN NUM. identifica aparatos de orientación y referencia.

Referentes

Seis referentes y las obras principales utilizadas para construir N29.

01

FERNANDO FLORES

Creando organizaciones para el futuro

Dolmen · 1997

Permite seguir compromisos durante un cambio. Ayuda a distinguir una tarea técnica terminada de una promesa que todavía requiere respuesta o reparación.

02

MARIO BUNGE

Emergencia y convergencia: novedad cualitativa y unidad del conocimiento

Gedisa · 2004

Exige una explicación que conecte mecanismos y resultados. Un indicador favorable no sustituye investigar qué produjo el comportamiento observado.

03

KENT BECK

Extreme Programming Explained

, Second Edition. Addison-Wesley · 2004

Aporta prácticas de cambios pequeños y verificación frecuente. Su utilidad aparece cuando permiten detectar y corregir antes, no sólo trabajar más rápido.

04

MARTIN FOWLER

Refactoring: Improving the Design of Existing Code

(2.ª ed.). Addison-Wesley · 2018

Relaciona integración, refactorización y evolución con cambios comprensibles. Ayuda a mantener capacidad de revisión sin congelar el sistema.

05

AMY EDMONDSON

Psychological Safety and Learning Behavior in Work Teams

Administrative Science Quarterly · 1999

Estudia condiciones para hablar de errores y dudas dentro de los equipos. Una advertencia sólo protege si puede escucharse y modificar la decisión.

06

WERNER VOGELS

Eventually Consistent

Communications of the ACM · 2009

Permite pensar compromisos de sistemas distribuidos y responsabilidades operativas. Ayuda a no suponer que todos los estados cambian juntos al instalar una versión.

N29 no resume estas fuentes ni las convierte en receta. Las pone en tensión para construir juicio profesional.

01 METSI · N29 PROBLEMA NÚCLEO

Pregunta profesional

¿Cómo ponemos un cambio en funcionamiento y qué hacemos si volver a la versión anterior ya no alcanza para resolver el problema?

Producto para el encuentro. Una ficha de cambio que separe artefacto, datos y efectos sobre personas, con dos ramas de recuperación y una comprobación de servicio recuperado en cada una.

Cómo recorrer la nota. Recorré identificación, autorización, recuperación y cierre operativo. El instrumento incluye una ficha didáctica con dos ramas; el ensayo y el cierre son núcleo porque sostienen la preparación. DevOps, procedencia y transferencia ofrecen contexto adicional; dentro del movimiento 3, procedencia amplía la ficha sin reemplazar sus controles de recuperación.

Revertir código no repara por sí solo los estados, dependencias y expectativas que ya cambiaron.

Volvió el programa anterior. Los pagos seguían duplicados

Imaginá una empresa de pagos ficticia. El equipo prepara una nueva versión del servicio que calcula comisiones y confirma transferencias. Las pruebas pasan y el cambio se habilita para una parte de las operaciones. Veinte minutos después, Atención empieza a recibir reclamos: algunas personas ven movimientos duplicados.

El equipo activa la vuelta a la versión anterior, conocida como rollback. El programa viejo queda funcionando otra vez. Pero los reclamos continúan. ¿Cómo puede ser, si ya se deshizo el cambio?

En realidad, sólo se deshizo una parte. La versión nueva había transformado datos. También había enviado mensajes que seguían esperando procesamiento. Y un proveedor estaba repitiendo solicitudes porque no había recibido una respuesta. El programa podía volver atrás; todo lo que había ocurrido alrededor no volvía con él.

Es como corregir el archivo de una invitación después de haberla enviado. El archivo queda bien, pero quienes recibieron la primera versión todavía tienen la fecha equivocada. Hay que identificar a esas personas y comunicar la corrección. Recuperar el archivo y reparar la consecuencia son acciones diferentes.

En la empresa de pagos, esa diferencia es mucho más delicada. No corresponde borrar movimientos ni restaurar una copia sin saber qué operaciones legítimas se perderían. El equipo necesita reconstruir cuáles se iniciaron, cuáles se confirmaron y qué efectos produjo cada una. La decisión requiere competencias técnicas, operativas y financieras dentro de sus responsabilidades.

Aparecen dos opciones. Una es restaurar un estado anterior y reconciliar lo ocurrido después. Otra es conservar el estado actual y corregir los registros afectados. Ninguna es buena por definición: depende de qué se sabe, qué puede recuperarse y qué daño adicional podría generar la intervención.

La empresa limita nuevas operaciones afectadas, conserva evidencia y prepara una reparación controlada. Las correcciones se verifican para no duplicar de nuevo un efecto. Los casos ambiguos no se resuelven adivinando. Se derivan a una revisión autorizada.

La investigación también encuentra un problema previo. La aprobación había mirado sobre todo el código y sus pruebas. No había preguntado suficientemente por datos, mensajes pendientes, configuración del proveedor ni capacidad de Atención para responder. La automatización había hecho lo programado; el plan era incompleto.

N29 trabaja sobre esa diferencia. Poner un programa en producción es una parte del cambio. La responsabilidad más amplia consiste en saber quién recibirá sus efectos, con qué evidencia se autoriza y cómo se recuperará el servicio si falla.

Hotel Horizonte: el botón de volver no desbloqueaba las habitaciones

HH-28 había mostrado que la marca general de accesibilidad no alcanzaba para comprobar condiciones específicas. También dejó escenarios de calidad que debían respetarse al corregir el sistema. HH-29 toma esas condiciones y prepara una nueva versión de la asignación.

Hotel Horizonte conserva una misma escena para que cambie el análisis y no el caso.

Federico despliega el cambio en una sede piloto. La cadena de construcción y pruebas termina sin errores. Poco después, Lucía informa que algunas habitaciones quedan bloqueadas. La pantalla técnica está en verde, pero el servicio no funciona como se esperaba.

El equipo revisa qué cambió. Una migración transformó estados del PMS, el sistema de gestión hotelera. Una regla de cerraduras ya utiliza la nueva interpretación. La planilla de Mariela sigue enviando «liberada» para indicar limpieza terminada. Ese uso no era incorrecto en sí mismo: N27 había separado limpieza de condición completa para asignar. El problema está en cómo conviven esas representaciones durante la transición.

Volver únicamente la aplicación puede empeorar la situación si la versión anterior interpreta mal los datos nuevos. Antes de hacerlo, Federico debe comprobar compatibilidad. Ricardo, por su parte, necesita saber qué procedimiento puede usar el turno mientras se investiga.

Camila confirma que la sede siguió vendiendo durante la ventana. No se habían detenido las obligaciones que el servicio debía atender. Por eso una copia anterior tampoco puede restaurarse sin considerar reservas y mensajes producidos después.

HOTEL HORIZONTE · N29

Seis responsabilidades dentro del sistema

El episodio cambia según la promesa, la operación, el dato y la autoridad que cada rol debe sostener.

DIRECCIÓN GENERAL

Elena Acosta

Define alcance, riesgo aceptable y continuidad de la promesa.

JEFATURA DE RECEPCIÓN

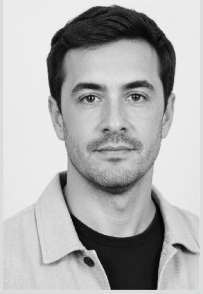
Lucía Ferreyra

Expone el episodio del huésped, las excepciones y la reparación.

GERENCIA DE OPERACIONES

Ricardo Sosa

Vuelve visibles capacidad, dependencias y condiciones de operación.



TECNOLOGÍA Y DATOS

Federico Müller

Reconstruye datos, contratos, integraciones y límites técnicos.

SUPERVISIÓN DE HOUSEKEEPING

Mariela Benítez

Aporta secuencias reales, tiempos, daños y condiciones de entrega.

GERENCIA COMERCIAL

Camila Duarte

Declara compromisos, demanda, canales y condiciones ofrecidas.

La respuesta limita la exposición del cambio a una franja de habitaciones y congela modificaciones comerciales que agregarían incertidumbre durante esa ventana. No borra las reservas ya tomadas ni suspende su atención. Las personas afectadas requieren una alternativa y una explicación de lo que ocurre.

Elena asigna a Ricardo autoridad para detener la exposición dentro del alcance acordado. Federico reconstruye versiones, configuración y transformaciones; Lucía verifica el resultado en Recepción; Mariela comprueba condiciones materiales. Las decisiones que exceden esas funciones deben escalarse a la competencia correspondiente.

El plan distingue tres acciones: volver una versión cuando es compatible, corregir datos hacia adelante cuando no conviene restaurarlos y reparar consecuencias que ya recibió una persona. No las reúne bajo una promesa vaga de «tenemos rollback».

Las primeras cincuenta transiciones serán una ventana de revisión, no una autorización automática para ampliar. Una discrepancia entre PMS y cerradura exige actuar antes. La ampliación requiere evidencia de los escenarios relevantes, no sólo que el contador llegue a cincuenta.

Tesis

Un cambio está bajo control cuando podemos responder cinco preguntas: **qué cambió, quién lo está usando, por qué se autorizó, quién puede detenerlo y cómo se recuperará el servicio**. Saber únicamente qué versión de código se instaló deja demasiadas consecuencias afuera.

El programa funciona junto con datos, parámetros, permisos, infraestructura y personas. Cambiar una regla de configuración puede alterar el resultado sin modificar una línea de código. Una migración puede dejar datos que una versión anterior ya no entiende. Una promesa enviada puede seguir vigente aunque se apague la función que la produjo.

Automatizar ayuda cuando hace repetibles los pasos y produce evidencia confiable. No convierte un plan incompleto en un plan correcto. Si la cadena no comprueba la compatibilidad con los datos o la capacidad del turno, un resultado favorable no demuestra esas propiedades.

Tampoco se resuelve agregando una firma manual a cada paso. Una aprobación aporta valor si alguien examina una pregunta relevante y puede cambiar la decisión. Si sólo copia el color del tablero, agrega espera sin necesariamente mejorar el control.

La entrega puede ser frecuente y cuidadosa a la vez. Cambios pequeños, pruebas pertinentes y recuperación preparada permiten aprender antes y localizar mejor los problemas. Pero la frecuencia no es un mérito aislado: liberar muchas veces mientras crecen las reparaciones nocturnas no demuestra una mejora del sistema.

La meta no es volver siempre atrás. Algunas consecuencias no se pueden deshacer. La meta es reconocer esa situación antes de exponer, limitar lo que haga falta y disponer de una reparación comprobable, sin borrar lo ocurrido ni descargarlo silenciosamente sobre Operaciones.

METSI · N29 · MAPA DE DECISIÓN

Volver atrás no siempre repara

Una liberación cambia más que el código.

Antes de exponer	Código, configuración, infraestructura, datos, terceros y operación. Probar el mismo artefacto que después se va a utilizar.
¿Es compatible volver?	Rollback sólo con compatibilidad comprobada de código y datos. Evaluar efectos externos y verificar la recuperación real del servicio.
Con datos transformados	Evaluar rollforward o restauración de datos con conciliación, si es viable. Comprobar compatibilidad, pérdidas y efectos antes de elegir la respuesta.
Con una promesa ya recibida	Aislar, comunicar, reparar o compensar. No se puede deshacer lo que el huésped ya vivió.

CONDICIONES QUE SE LEEN EN CONJUNTO

Límite del piloto

Cincuenta transiciones o la primera discrepancia. Ricardo detiene; Lucía comprueba el resultado antes de ampliar.

Las relaciones orientan la decisión; no reemplazan la explicación del texto.

05 METSI · N29 DESARROLLO EXTENSIÓN

De la calidad esperada a la decisión de liberar

N28 definió qué necesitaba demostrar el servicio. N29 convierte esas condiciones en un proceso concreto de cambio. Vincula pruebas y decisiones con una versión, un entorno y un alcance de uso.

Conviene separar tres momentos. **Integrar** combina trabajo en una base compartida. **Desplegar** instala un artefacto en un ambiente. **Liberar** habilita una capacidad para un uso determinado. Pueden ocurrir muy cerca, pero responden preguntas diferentes.

La separación tampoco vuelve inocuo el despliegue. Un programa instalado con su función visible apagada puede ejecutar tareas de fondo, consumir recursos o transformar datos. Hay que examinar todos sus efectos, no sólo si apareció un botón nuevo.

N30 observará lo que ocurra después. Para interpretar sus señales necesitará saber qué combinación de versiones y condiciones recibió cada población. Esa información empieza a construirse aquí, no cuando aparece el primer incidente.

06 METSI · N29 DESARROLLO EXTENSIÓN

Las ideas que sostienen el recorrido

Humble y Farley desarrollan la entrega continua como capacidad de mantener software preparado para liberarse mediante un proceso confiable. Beck y Fowler ayudan a comprender por qué integrar, probar y modificar en pasos pequeños reduce la acumulación de diferencias difíciles de revisar.

Forsgren, Humble y Kim, junto con la investigación DORA, relacionan prácticas de entrega con desempeño y estabilidad. Sus resultados no significan que cualquier organización deba copiar una frecuencia ni que una correlación apruebe un cambio concreto. Sirven para estudiar cómo se entrega y se aprende.

NIST aborda desarrollo seguro y cadena de suministro. SLSA organiza garantías sobre origen y construcción. Esas fuentes hacen preguntas que una prueba funcional no responde: de dónde vino el artefacto, cómo se produjo y qué confianza merece su historia.

Las fuentes de gestión de servicios y seguridad aportan responsabilidades y controles. OpenGitOps describe principios para gestionar estados declarados mediante versiones y reconciliación. Git y GitHub aportan mecanismos diferentes para historia y colaboración, que veremos sin confundirlos con el gobierno completo del cambio.

Edmondson permite examinar si alguien puede advertir una falla y detener una liberación sin que la presión lo silencie. Flores, Bunge y García ayudan a relacionar esa práctica con compromisos, explicaciones y condiciones organizacionales. Una cadena técnica puede estar bien diseñada y fallar si nadie puede discutir lo que está por hacer.

Movimiento 1 · Saber exactamente qué estamos cambiando

Si dos personas dicen «estamos usando la versión nueva», todavía falta saber de qué hablan. Pueden compartir aplicación y tener parámetros, permisos o datos distintos. Antes de aprobar necesitamos identificar la combinación que produce el comportamiento.

Integración continua: encontrar diferencias cuando todavía son pequeñas

Pensá en un trabajo grupal. Si cada integrante escribe durante un mes sin mirar a los demás, al juntar todo aparecen definiciones distintas, repeticiones y partes incompatibles. Revisar pequeños avances juntos permite detectar antes esos problemas.

METSI · N29 / LECTURA VISUAL

Integrar para detectar pronto

01 La **integración continua**, o CI, mantiene cambios pequeños incorporados y verificados con frecuencia en una base compartida. No significa solamente tener una herramienta que ejecuta pruebas. Requiere que el trabajo se integre realmente y que las fallas reciban atención.

02 El mecanismo es sencillo: cuanto menos tiempo permanecen separados los cambios, menos diferencias se acumulan antes de contrastarlas. Si una verificación falla después de un cambio pequeño, suele ser más fácil localizar qué lo produjo. No es una garantía de ausencia de errores; es una forma de reducir demora y ambigüedad del diagnóstico.

03 En HH-29, modificar la interpretación de un estado exige revisar pruebas del intercambio y de los consumidores afectados. Si sólo se comprueba que el código compila, la integración puede pasar y conservar el error de significado.

04 Una base con verificaciones fallidas necesita una política: corregir, retirar el cambio o registrar una excepción justificada con control. Acumular cambios sobre un estado que nadie confía vuelve menos útil cada resultado siguiente.

También hay que investigar las pruebas intermitentes. Si una prueba pasa y falla sin una explicación conocida, no conviene resolverlo repitiéndola hasta obtener verde. Puede haber un problema del producto, del ambiente o de la propia prueba. Distinguirlo recupera la confianza en la señal.

El resultado de CI prepara evidencia. No autoriza por sí solo una exposición a huéspedes. Esa decisión agrega preguntas sobre datos, operación, riesgos y recuperación.

CI/CD: preparado para liberar no es lo mismo que liberar automáticamente

La sigla CD se usa para dos ideas relacionadas. **Entrega continua**, continuous delivery, significa mantener los cambios en condiciones de poder liberarse mediante un proceso confiable. Puede conservar una decisión explícita antes de producción. **Despliegue continuo**, continuous deployment, lleva automáticamente a producción los cambios que superan los controles definidos.

La diferencia importa. Un equipo puede practicar entrega continua sin poner cada modificación inmediatamente al alcance de todos. Puede necesitar una ventana coordinada, una habilitación gradual o una revisión del riesgo.

Una **cadena automatizada**, también llamada pipeline, reúne pasos como construir, probar, verificar y desplegar. Automatiza una secuencia acordada. Su capacidad depende de las condiciones que representa y de la confianza en sus herramientas, permisos y resultados.

En el hotel, la cadena puede preparar y comprobar el artefacto. La habilitación de una nueva regla puede esperar a que estén disponibles el procedimiento de Recepción y la compatibilidad de datos. No es una contradicción: entrega preparada y exposición autorizada son estados distintos.

Si se opta por despliegue continuo, la autoridad no desaparece. Se expresa en reglas, permisos, criterios y mecanismos de detención previamente establecidos. Quienes los definen necesitan revisar si siguen siendo apropiados. «Fue automático» describe cómo ocurrió, no quién respondió por la política.

Tampoco se debe considerar manual igual a inseguro. Un cambio infrecuente puede usar pasos manuales claros y verificables. La pregunta es qué riesgo de variación, error y demora introduce cada método, y cómo se controla. La automatización es una herramienta para mejorar esa capacidad, no un objetivo que justifica cualquier exposición.

Git y GitHub: historia por un lado, colaboración por otro

Git registra versiones de archivos y relaciones entre cambios. Permite comparar, recuperar y trabajar con líneas de desarrollo. GitHub es una plataforma que utiliza repositorios y agrega colaboración, revisiones y automatizaciones. No son dos nombres de lo mismo.

En Git, un commit identifica una revisión registrada. Una rama permite trabajar sobre una línea de cambios y una etiqueta puede señalar un punto relevante. Esta historia ayuda a reconstruir diferencias, pero no demuestra que cada cambio haya sido correcto o aprobado. Pro Git, Chacon y Straub.

Una solicitud de cambio en GitHub puede reunir la diferencia propuesta, su explicación, pruebas y comentarios. Las protecciones de rama permiten exigir controles o revisiones y restringir ciertas operaciones; deben configurarse de acuerdo con la política que se quiere sostener. Documentación de ramas protegidas.

En HH-29, la propuesta necesita explicar qué cambia en la asignación y qué riesgo trata. Federico revisa implementación; Lucía ayuda a contrastar el comportamiento con la atención; la autoridad correspondiente evalúa exposición. No todos tienen que revisar el mismo detalle ni hacer de expertos en todo.

La revisión agrega valor cuando puede producir una objeción que cambie algo. Dos aprobaciones rápidas de personas sin contexto pueden ofrecer menos control que una revisión pertinente. La cantidad de firmas no mide automáticamente independencia o calidad.

La historia también necesita límites. Una configuración cambiada directamente en producción puede alterar el servicio sin aparecer en el repositorio. Por eso hay que conectar lo registrado con lo que realmente está funcionando. Y no corresponde guardar credenciales en archivos versionados para mejorar la trazabilidad: se registra su referencia y gestión, no el secreto expuesto.

Artefacto: identificar la unidad que realmente se instala

Una receta no es la torta. El código fuente describe qué construir; el **artefacto** es la unidad resultante que se distribuye o instala, como un paquete o una imagen de aplicación. Entre ambos intervienen herramientas y dependencias.

Si se reconstruye para cada ambiente, pueden introducirse diferencias. Una biblioteca descargada después podría tener otro contenido o una opción de construcción podría cambiar. Por eso es valioso construir una vez, identificar el resultado y promover esa misma unidad por los controles correspondientes.

Un identificador por contenido, como un hash, ayuda a comprobar que dos copias tienen el mismo contenido bajo el mecanismo utilizado. No identifica por sí solo a un autor confiable ni prueba que el programa sea correcto. Una firma y su cadena de confianza responden otras preguntas.

El artefacto candidato a liberación se relaciona con fuente, dependencias, proceso de construcción y resultados de prueba. «Está listo» significa listo para un alcance y unas condiciones, no libre de todo defecto imaginable.

En el hotel, el registro debe permitir verificar que la unidad desplegada corresponde a la examinada. Si se descarga parte del sistema en tiempo de ejecución, esa dependencia también requiere identificación y control. No se presenta todo como inmutable cuando una parte puede cambiar fuera del artefacto.

Promover la misma unidad no produce necesariamente el mismo comportamiento. Datos, configuración y permisos siguen importando. La identidad del artefacto es un punto de apoyo para la investigación, no una descripción completa del sistema.

Ambiente y configuración: lo que cambia aunque el código sea igual

Dos teléfonos del mismo modelo pueden comportarse distinto si tienen permisos, conexión y ajustes diferentes. Del mismo modo, una aplicación depende de las condiciones en que se ejecuta.

El **ambiente** incluye infraestructura, red, servicios, datos y capacidades disponibles. La **configuración** establece parámetros y opciones de comportamiento. Un límite de espera, una regla de cerradura o una bandera de funcionalidad pueden cambiar el resultado sin modificar el código principal.

HH-29 necesita conocer tanto la aplicación como la regla que interpreta un estado para emitir acceso. Si una cambia y la otra no, el comportamiento conjunto puede dejar de ser válido. Probar cada componente por separado no necesariamente revela esa incompatibilidad.

Prueba y producción nunca son idénticas en todo. La tarea consiste en identificar diferencias relevantes: volumen de datos, permisos, demoras externas, versiones, tareas de fondo y patrones de uso. Se eligen ensayos que examinen los mecanismos que esas diferencias podrían afectar.

Los datos de ensayo deben prepararse con protección adecuada. No se copia información personal sin necesidad y sin condiciones de uso válidas sólo para conseguir realismo. Datos sintéticos bien diseñados pueden representar muchas excepciones; cuando no alcanzan, cualquier prueba con condiciones reales requiere alcance, autorización y cuidado específicos.

La **deriva** aparece cuando el estado real se aparta de la definición controlada. Puede deberse a una emergencia legítima o a una modificación no autorizada. Detectarla no implica sobrescribir a ciegas. Primero se determina qué ocurrió y qué acción segura corresponde; después se reconcilia la situación con la fuente gobernada.

Infraestructura como código: poder reconstruir, no sólo describir

Si instalar un ambiente depende de recordar una secuencia de clics, repetirlo durante un incidente puede ser difícil. La **infraestructura como código** expresa recursos y reglas mediante definiciones que pueden revisarse y ejecutarse.

Eso permite comparar qué se propone cambiar y conservar su historia. Pero una definición válida puede crear un recurso equivocado, cortar una conexión necesaria o eliminar datos. La revisión necesita mirar el efecto, no sólo que la sintaxis sea correcta.

Antes de aplicar, el equipo examina qué se creará, modificará o retirará. Las operaciones destructivas requieren un tratamiento especial. El alcance aprobado debe coincidir con el plan y con lo que después se observa.

La reconstrucción se ensaya en un entorno controlado. Si faltan permisos, secretos o pasos manuales, se corrige el procedimiento o se documentan excepciones concretas. La mera existencia de archivos no demuestra que otra persona pueda levantar el servicio dentro del tiempo necesario.

GitOps agrega una forma particular de gestionar el estado deseado: declararlo, mantenerlo versionado, obtenerlo automáticamente desde la fuente y reconciliarlo con el estado observado. No es sinónimo de cualquier uso de Git ni de toda infraestructura como código. Principios OpenGitOps.

La reconciliación también necesita límites operativos. Una corrección de emergencia no debe ser deshecha de manera peligrosa por un controlador que desconoce su razón. Se diseñan procedimientos de suspensión, excepción y retorno. Automatizar el estado equivocado puede repetir el error con mucha consistencia.

08 METSI · N29 DESARROLLO EXTENSIÓN

DevOps: construir y operar sin pasarse el problema

Imaginá que un equipo entrega un equipo de sonido y se retira cuando se enciende. Quienes organizan el evento descubren después que nadie sabe resolver una interferencia. La entrega técnica ocurrió; la capacidad de uso quedó incompleta.

DevOps aborda esa separación entre desarrollo y operación. Reúne prácticas de colaboración, automatización, retroalimentación y aprendizaje para poder cambiar y sostener servicios. No queda definido por comprar una herramienta ni por llamar DevOps a un puesto.

En HH-29, Federico necesita conocer qué ve Lucía ante un resultado desconocido y qué puede hacer el turno. Operaciones participa antes de recibir la versión, ayudando a definir pruebas y recuperación. La información vuelve al diseño en lugar de quedar como una queja posterior.

Esa colaboración no borra competencias ni separaciones necesarias. Quien desarrolla no debe tener por defecto toda autoridad para aprobar, modificar producción y eliminar evidencia. Compartir responsabilidad por el resultado requiere roles claros, no acceso ilimitado de todos a todo.

El beneficio se observa en el recorrido: menos espera inútil, diagnóstico más rápido, cambios comprensibles y recuperación efectiva. No se demuestra sólo con frecuencia de despliegues. Si cada versión transfiere horas de conciliación a Recepción, el trabajo total puede estar empeorando.

La perspectiva de *The DevOps Handbook* y *Accelerate* ayuda a mirar flujo y aprendizaje. La de Edmondson agrega una condición organizacional: alguien debe poder decir que el cambio no está listo o que el resultado no coincide. Sin esa posibilidad, las herramientas pueden acelerar un silencio peligroso.

Movimiento 2 · Decidir quién recibe el cambio y bajo qué condiciones

Hasta aquí identificamos lo que cambia. Ahora toca decidir cómo se usa. No todos los cambios requieren el mismo proceso, pero todos necesitan un alcance y una forma de responder si producen un resultado no previsto.

Dos registros del trabajo real permiten contrastar la decisión con sus condiciones de operación.

Aprobación según riesgo: revisar lo importante, no contar firmas

Cambiar un texto informativo y cambiar la regla que autoriza una llave pueden requerir muy distinto cuidado, aunque ambos ocupen una sola línea. El tamaño del archivo no mide la consecuencia.

Una **aprobación basada en riesgo** considera población, datos, efectos externos, dificultad de detectar la falla y posibilidad de recuperarse. Con esa información define controles y autoridad. No significa que todo cambio pequeño pase automáticamente ni que todo cambio grande necesite un comité.

Una **puerta de decisión** es el punto donde se evalúa si se cumplen condiciones para asumir el siguiente compromiso. Puede estar parcialmente automatizada. Lo esencial es que tenga criterios, resultados posibles y responsables: avanzar, limitar, pedir más evidencia o rechazar.

En HH-29, los escenarios de N28 forman parte de esa puerta. Si la prueba sólo demuestra que el PMS acepta el estado, falta comprobar la relación con cerraduras, equipamiento y turno. La fecha prevista no reemplaza esa evidencia.

Un riesgo conocido puede requerir una limitación o una excepción autorizada, siempre dentro de las obligaciones aplicables. La excepción debe decir qué cambia, cuánto dura y cómo se observa. No sirve un campo «acepto el riesgo» que nadie relaciona con una consecuencia.

También se revisan los controles. Que un control nunca haya bloqueado un cambio no demuestra por sí solo que sea inútil: podría estar previniendo problemas o cubrir un riesgo poco frecuente. Hay que examinar su propósito y evidencia antes de retirarlo. Del mismo modo, un control que sólo reproduce información ya disponible puede rediseñarse para reducir espera.

Separar funciones sin impedir una respuesta urgente

La **segregación de funciones** evita que una sola persona pueda completar sin control una combinación peligrosa de acciones. Por ejemplo, proponer un cambio, aprobarlo, ejecutarlo y borrar los rastros sin que nadie pueda advertirlo.

No exige convertir cada operación en una peregrinación de firmas. Se diseña sobre riesgos: permisos mínimos, revisión pertinente, identificación de quien actúa y protección de la evidencia. Dos firmas de personas que no pueden ver ni cuestionar el cambio no crean una separación efectiva.

HH-29 distingue propuesta técnica, verificación operativa y aceptación del alcance. Lucía no tiene que auditar cada biblioteca para comprobar que Recepción puede interpretar y reparar una reserva. Federico no puede declarar resuelta esa capacidad sólo porque compiló el programa.

Un equipo pequeño puede necesitar controles compensatorios. En una urgencia autorizada, por ejemplo, acceso temporal limitado, observación y registro reforzados pueden acompañar una intervención que no admite toda la secuencia ordinaria. Su suficiencia depende del caso; no es una autorización genérica para saltear controles obligatorios.

Después se verifica lo actuado, se reconcilia la configuración y se retiran los permisos extraordinarios, no los permisos permanentes necesarios para trabajar. La revisión posterior no convierte retroactivamente cualquier acción en legítima: examina lo ocurrido y sus consecuencias.

Si las emergencias se vuelven habituales, hay que revisar el proceso normal. Tal vez sea demasiado lento, falte capacidad o se estén clasificando urgencias para evitar preguntas. Mantener privilegios excepcionales sin límite no resuelve ninguna de esas causas.

Exposición progresiva: aprender sin extender de golpe el problema

Una nueva regla puede instalarse y habilitarse primero en un alcance limitado. Una **cohorte** es el conjunto seleccionado para esa etapa. Un **canario** es una exposición inicial reducida que se observa antes de ampliar. Una **bandera de funcionalidad** permite habilitar o deshabilitar un comportamiento bajo condiciones definidas.

Los mecanismos no son equivalentes ni suficientes por sí solos. Una bandera apagada puede no detener tareas de fondo. Un canario pequeño puede usar la misma base y afectar indirectamente a personas que no pertenecen a su cohorte. La frontera real del impacto se investiga, no se supone.

En HH-29 se elige una franja de habitaciones con soporte y condiciones conocidas. Antes se comprueban riesgos relevantes en ensayos protegidos. No se usa a una población vulnerable como material de prueba ni se omiten todos los casos difíciles para conseguir resultados favorables.

La ampliación se apoya en señales técnicas, operativas y de experiencia. Una tasa baja de errores no habilita avanzar si aparece una asignación incompatible o un procedimiento manual que el turno no puede sostener. Los criterios se definen antes para que la presión de calendario no los cambie silenciosamente.

El tiempo de observación también necesita razón. Hay patrones que no aparecen en unos minutos y dependencias que se utilizan poco. Cincuenta transiciones pueden permitir una primera revisión, pero no demostrar cobertura de todo. Si el resultado es inconcluso, se mantiene o reduce alcance mientras se investiga.

La exposición gradual no siempre es técnicamente posible tal como se desea. Un cambio compartido puede requerir compatibilidad adicional, una migración por etapas o una ventana coordinada. En lugar de fingir un canario aislado, se reconoce esa restricción y se prepara el control correspondiente.

Cambiar en pasos pequeños suele facilitar comprensión y recuperación, pero no elimina dependencias compartidas.

Movimiento 3 · Volver, corregir y reparar son acciones distintas

Antes de liberar, el equipo necesita saber qué cambios puede deshacer y cuáles dejan efectos. No es pesimismo: es una condición para elegir rápido y con información cuando aparece un problema.

Rollback: volver una parte a un estado anterior

Un **rollback** devuelve una versión o un estado al anterior dentro de un alcance definido. Puede referirse sólo a la aplicación. No es incorrecto llamarlo rollback de código; lo incorrecto es prometer que por eso quedó recuperado todo el servicio.

Volvamos a la invitación. Recuperar el archivo anterior es posible. Retirar de la memoria de todos la fecha que leyeron, no. El efecto externo necesita otra acción. Lo mismo ocurre con una llave emitida, una reserva comunicada o un cobro confirmado.

El plan de vuelta pregunta primero si la versión anterior entiende los datos actuales y puede convivir con las dependencias que ya cambiaron. Si no, reinstalarla puede agregar errores. Tener guardado el binario es necesario para ciertas reversiones, pero no demuestra que usarlas sea seguro.

Las copias de respaldo también necesitan prueba. Hay que saber qué contienen, si se pueden restaurar y qué operaciones posteriores quedarían afuera. Una restauración que recupera ayer y pierde reservas válidas de hoy no resuelve automáticamente la situación.

HH-29 separa el tiempo técnico de volver la aplicación del tiempo hasta recuperar el servicio y atender consecuencias. Federico puede terminar su paso mientras Lucía todavía revisa reservas. Ambos tiempos importan, pero no describen lo mismo.

La decisión puede ser no volver. Si el estado actual está mejor comprendido y una corrección acotada protege más, puede convenir avanzar. El criterio es recuperar una condición segura y explicable, no cumplir la promesa de usar el botón que se había preparado.

Migrar datos sin perder la posibilidad de convivir

Cambiar el significado de un estado es más difícil que cambiar su nombre. Mientras distintas versiones conviven, cada una debe interpretar correctamente los datos que recibe.

Una estrategia es **expandir y contraer**. Primero se agrega una representación nueva compatible con el funcionamiento existente. Después se adaptan lectores y escritores, se migran datos con controles y, cuando hay evidencia suficiente, se retira la forma antigua. El orden depende del sistema; no es una receta idéntica para cualquier base.

En el hotel podría mantenerse la señal de limpieza mientras se incorpora una representación más completa de condiciones para asignar. No corresponde traducir «limpia» como «asignable» sólo para evitar un cambio en el consumidor. Esa traducción inventaría una garantía que la señal original no contiene.

Si durante una transición se escribe en dos representaciones, aparece otro riesgo: que diverjan. Hay que definir cuál decide en cada situación, cómo se detectan diferencias y cómo se reconcilian. La doble escritura no es segura por el solo hecho de existir.

El plan también marca cuándo deja de ser posible una vuelta sencilla. Antes de retirar un campo, una versión anterior puede seguir leyendo. Después de eliminarlo o cambiar irreversiblemente su significado, quizá ya no. Ese punto debe conocerse antes de ejecutar.

El ensayo usa datos controlados y recorre también casos ambiguos: escritura parcial, consumidor antiguo y mensaje demorado. No se fuerza una pérdida de reservas reales para comprobar si el procedimiento funciona. La prueba debe reducir incertidumbre sin crear un daño innecesario.

Rollforward y reparación: corregir desde donde estamos

El **rollforward** introduce una corrección hacia adelante cuando volver no es la mejor opción o ya no restituye un estado válido. No significa seguir desplegando hasta que algo funcione. Necesita diagnóstico, alcance y comprobaciones.

En HH-29 puede requerirse corregir registros transformados, adaptar una regla y reconciliar mensajes pendientes. Primero se identifica el conjunto afectado. Los casos claros pueden seguir una operación preparada; los ambiguos necesitan revisión. No se transforma toda la base porque aparecieron algunos bloqueos.

Una operación de reparación debe poder repetirse de forma segura cuando sea posible. La idempotencia, trabajada en N27, ayuda a evitar que un reintento vuelva a producir el mismo efecto. Su alcance debe quedar definido: una corrección idempotente en la base no asegura que una notificación externa no se envíe dos veces.

Se compara antes y después, se verifica con el turno y se detiene si aparece otra interpretación no prevista. Las copias y registros necesarios se protegen con acceso y conservación adecuados. Guardar todo indiscriminadamente también puede crear riesgos.

La **reparación a la persona** es más amplia que la corrección técnica. Si recibió una promesa equivocada, necesita conocer su situación y una respuesta dentro de las responsabilidades aplicables. No se declara reparada porque un campo volvió a su valor correcto.

Después, la solución de emergencia vuelve al proceso controlado: revisión, pruebas, configuración y documentación. Si queda como una secuencia privada que sólo recuerda una persona, la próxima liberación puede repetir el problema o deshacer la corrección sin saberlo.

Procedencia: poder explicar de dónde salió lo que funciona

Cuando un alimento tiene un problema, interesa conocer su lote y origen. En software, la **procedencia** conecta una unidad con su fuente, dependencias y proceso de construcción. Ayuda a investigar qué recibió una población y qué otras versiones pueden estar afectadas.

Una **SBOM**, lista de materiales de software, describe componentes conocidos del producto. Una **firma** permite comprobar propiedades de integridad y autoría bajo una cadena de confianza. Una **atestación** declara hechos sobre un proceso o artefacto y puede verificarse según su formato y confianza. Responden preguntas diferentes.

La lista no demuestra que cada componente sea seguro. La firma no demuestra que el uso sea adecuado. La atestación necesita evaluar a quien la produce y aquello que realmente acredita. Autenticar un artefacto defectuoso permite saber mejor qué ocurrió, pero no elimina el defecto.

SLSA 1.2 organiza niveles y rutas para mejorar garantías de cadena de suministro, incluyendo fuente, construcción y formatos de atestación. Se utiliza aquí como marco de procedencia, no como certificación de toda la calidad del hotel. Especificación SLSA 1.2.

NIST SP 800-204D estudia cómo incorporar seguridad de cadena de suministro en procesos CI/CD. Su aporte amplía la mirada a herramientas y pasos de construcción y entrega. No basta con examinar el código principal si el entorno que lo transforma o distribuye está comprometido. NIST SP 800-204D.

La investigación va en dos sentidos. Desde la fuente, identifica qué se construyó. Desde la operación, identifica qué artefacto y condiciones estaban activos. Una firma válida de un archivo que nunca llegó a esa sede no explica su incidente.

Un aviso de vulnerabilidad pone a prueba la historia

Supongamos, como ejercicio, que se informa una vulnerabilidad en una biblioteca. El equipo no debería responder únicamente «usamos esa tecnología» o «todavía no hubo incidentes».

Primero identifica qué artefactos incluyen la versión afectada y qué evidencia respalda el inventario. Después localiza dónde están desplegados y bajo qué configuración. También verifica dependencias dinámicas que podrían no aparecer en una lista desactualizada.

La evaluación especializada determina exposición, condiciones de explotación y medidas apropiadas. La severidad publicada orienta atención, pero no reemplaza el contexto. A la inversa, no conocer una explotación local no demuestra ausencia de riesgo.

La respuesta puede ser actualizar, limitar una función, aislar o retirar, según el caso. La alternativa elegida necesita conocer sus propias consecuencias: apagar una dependencia puede cortar una capacidad crítica. Seguridad y continuidad deben coordinarse, no resolverse mediante decisiones aisladas.

El registro de procedencia reduce el alcance de la búsqueda y permite una respuesta más precisa. Si no se sabe qué se instaló, primero habrá que producir esa información. La incertidumbre no debe transformarse en una falsa afirmación de que nadie está expuesto.

La escena documental devuelve la estrategia a una situación de operación, decisión y reparación.

Instrumento HH-29: el plan de cambio y recuperación

La ficha conecta doce decisiones. Su utilidad se prueba cuando otra persona puede reconstruir el cambio y reconocer qué hacer ante una falla, sin depender de explicaciones privadas del autor.

Hotel Horizonte conserva una misma escena para que cambie el análisis y no el caso.

Su utilidad se prueba cuando otra persona puede reconstruir el cambio y reconocer qué hacer ante una falla, sin depender de explicaciones privadas del autor.

01 **Cambio y alcance:** qué comportamiento se modifica, para quién y qué queda fuera. Se incluyen efectos indirectos previsibles, no sólo usuarios del botón nuevo.

02 **Artefacto:** qué unidad se instala y cómo se relaciona con fuente, dependencias y pruebas. Se distingue identidad de corrección.

03 **Ambientes:** qué diferencias entre ensayo y uso real podrían cambiar el resultado.

04 **Configuración:** qué parámetros y permisos intervienen. Se registran referencias y gestión de secretos, no credenciales expuestas en la ficha.

05 **Datos:** qué se transforma, qué versiones podrán leerlo y qué ocurrirá con operaciones durante la ventana.

06 **Infraestructura:** qué recursos cambian y cómo se verificará su estado sin asumir que una ejecución exitosa demuestra servicio.

07 **Contratos:** qué significados, formatos, tiempos y respuestas deben conservarse entre participantes.

08 **Evidencia:** qué escenarios de calidad se comprobaron, cuáles fallaron y qué incertidumbres siguen abiertas.

09 **Aprobación:** quién autoriza qué alcance y dentro de qué competencia. Se conserva la razón, no sólo el nombre.

10 **Exposición:** cómo se habilita el cambio, qué se observa y qué señal detiene o impide ampliar.

11 **Recuperación:** cuándo conviene volver, cuándo corregir hacia adelante y cómo se atenderán consecuencias externas.

12 **Cierre:** qué demuestra que la operación puede sostener la versión y qué apoyos temporarios siguen siendo necesarios.

HOTEL HORIZONTE · N29

Seis responsabilidades dentro del sistema

El episodio cambia según la promesa, la operación, el dato y la autoridad que cada rol debe sostener.

DIRECCIÓN GENERAL

Elena Acosta

Define alcance, riesgo aceptable y continuidad de la promesa.

JEFATURA DE RECEPCIÓN

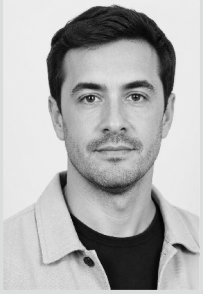
Lucía Ferreyra

Expone el episodio del huésped, las excepciones y la reparación.

GERENCIA DE OPERACIONES

Ricardo Sosa

Vuelve visibles capacidad, dependencias y condiciones de operación.



TECNOLOGÍA Y DATOS

Federico Müller

Reconstruye datos, contratos, integraciones y límites técnicos.

SUPERVISIÓN DE HOUSEKEEPING

Mariela Benítez

Aporta secuencias reales, tiempos, daños y condiciones de entrega.

GERENCIA COMERCIAL

Camila Duarte

Declara compromisos, demanda, canales y condiciones ofrecidas.

Completada para HH-29, la ficha no dice sólo «desplegar asignable». Identifica qué representación se incorpora, cómo convive con «liberada», qué regla de cerradura participa y qué reserva puede estar afectada. La unidad de cambio conserva relaciones, no pretende reunirlo todo en un único archivo.

El árbol de recuperación empieza por preguntas observables. ¿La versión anterior entiende los datos actuales? ¿Se emitieron accesos o mensajes externos? ¿Existe una corrección confiable para el conjunto afectado? ¿Qué casos permanecen ambiguos? Cada respuesta conduce a una acción autorizada y una comprobación posterior.

Si el equipo no puede responder, la ficha no está lista para justificar una ampliación. Puede necesitar más prueba, menor alcance o una protección diferente. No se completa la incertidumbre con «se resolverá en operación».

La revisión cruza la ficha con lo que realmente se ejecutó. Si el artefacto coincide pero una regla se cambió fuera de la secuencia, hay una diferencia que investigar. Si el alcance aprobado era una franja y la configuración habilitó toda la sede, el control falló aunque nadie haya informado todavía un incidente.

Ficha completada · Una franja de habitaciones y dos recuperaciones

Ejemplo didáctico construido a partir de HH-29. Los rótulos ASIG-A/B, E1/E2, C1/C2 y H-201/202 sólo identifican piezas del ensayo. No son versiones ni comprobaciones reales.

Cambio, artefacto y alcance

ASIG-B incorpora la comprobación de requisitos para asignar. Se ensaya sólo con H-201 y H-202 y reservas de prueba. ASIG-A es la aplicación anterior. En una ejecución real cada rótulo se vincularía con su contenido identificado y sus pruebas.

Datos, configuración y contrato

E2 incorpora condiciones específicas y conserva «liberada» como limpieza; C2 es la regla de cerradura compatible. No se convierte limpieza en autorización. La ficha registra qué lectores aceptan E1/E2 y qué mensajes quedan pendientes.

Ambiente, infraestructura y evidencia

Entorno aislado de ensayo; sin cambio de infraestructura en este ejemplo. Se prueban reserva compatible, requisito ausente, mensaje tardío y migración parcial. El paso a operación queda pendiente hasta documentar resultados y diferencias con el entorno real.

Autoridad y detención

Elena define alcance; Ricardo puede detener la exposición; Federico verifica la combinación; Lucía y Mariela comprueban atención y condiciones. Una discrepancia PMS–cerradura impide ampliar antes de completar las cincuenta transiciones.

La elección depende del estado comprobado

Rama	Condición y acción	Cómo reconocer recuperación
A · Volver una versión	El ensayo demuestra que ASIG-A interpreta los datos actuales y convive con la configuración elegida. Se identificaron y reconciliaron efectos y mensajes. Reinstalar A sólo dentro de ese alcance verificado.	La reserva de prueba conserva requisitos y una única asignación válida; PMS y acceso concuerdan. Lucía completa el recorrido y verifica que no queden efectos pendientes ocultos.
B · Corregir hacia adelante	A no interpreta E2 o dejaría una condición ambigua. Mantener exposición limitada; identificar registros afectados; aplicar una corrección B1 y reconciliar mensajes. Los casos desconocidos van a revisión autorizada.	Comparar antes/después de cada registro; repetir la reparación sin duplicar el efecto protegido; comprobar habitación, asignación y acceso con el turno. La atención a la persona conserva seguimiento propio.

Cierre. Si no puede probarse ninguna rama, no se amplía: se mantiene una alternativa segura y se investiga. Un rollback rápido no cierra la reparación; una rama ensayada tampoco demuestra todos los turnos o dependencias.

Un rollback rápido no cierra la reparación.

Ensayar una recuperación antes de necesitarla

Ensayo resuelto ·doce reservas ficticias. En un ambiente de prueba se libera una nueva regla de asignación y se transforman doce reservas: nueve quedan correctas, dos no pueden leerse con la versión anterior y una ya produjo una llave equivocada. Son tres conjuntos disjuntos. La señal para detener la ampliación es cualquier reserva cuyo estado no pueda explicarse o una llave emitida sin asignación válida.

Primera decisión: no volver por reflejo. Federico comprueba que el programa anterior no interpreta las dos reservas transformadas. Volver sólo el código dejaría esos registros ilegibles. Ricardo mantiene el piloto detenido y autoriza una corrección acotada hacia adelante; conserva copia, versión, configuración y lista de casos.

Segunda decisión: separar servicio de efecto externo. Se corrigen y verifican las dos reservas bloqueadas, una sola vez por identificador. La llave equivocada no se arregla cambiando un campo: se invalida, se verifica el acceso y Recepción contacta a la persona afectada. Lucía confirma que las nueve reservas restantes siguen siendo utilizables. Nadie marca «recuperado» por el mero éxito del despliegue.

Comprobación de salida. Las doce reservas deben tener estado explicable; las dos corregidas pasan una nueva lectura; la llave errónea ya no abre y la persona recibió una respuesta. Se registran por separado el tiempo técnico, el tiempo de servicio y la reparación. Si falta evidencia de la llave, se mantiene la protección y no se amplía el cambio.

Qué aprendió el ensayo. El plan decía «volver a la versión previa», pero esa rama era inviable después de transformar datos. Antes de otra liberación, el equipo debe probar convivencia de versiones, conservar una ruta de corrección y definir quién repara efectos fuera del sistema. La decisión se revisa con N27, N28 y N30.

Para transferir el método, repetí el ensayo con una condición distinta: la versión anterior sí lee los datos y todavía no hay llaves ni avisos externos. Ahí un rollback probado puede ser la rama segura. Escribí qué observarías antes de elegirla y qué prueba permitiría cerrar el incidente sin confundir versión instalada con servicio recuperado.

Cerrar la liberación sin retirar el apoyo demasiado pronto

Una versión puede funcionar bien durante el piloto porque hay especialistas adicionales, vigilancia manual y menos demanda. Antes de retirar ese apoyo hay que comprobar qué depende de él.

El **cierre de liberación** confirma que la versión puede sostenerse bajo las condiciones previstas. Incluye señales, permisos, procedimientos, responsabilidades y tratamiento de pendientes. No coincide necesariamente con terminar el pipeline o habilitar todo el tráfico.

Si Lucía sólo puede resolver un bloqueo llamando a Federico, todavía existe una dependencia que debe reconocerse. Puede ser un apoyo temporal autorizado, una capacidad que deba organizarse de manera permanente o una razón para limitar el uso. No se elimina del registro para declarar terminado el proyecto.

Las tareas pendientes se distinguen por consecuencia. Una mejora conveniente puede pasar a la cartera. Una falla que impide detectar una asignación incorrecta puede bloquear la ampliación. Llamarlas a todas «deuda» no permite decidir.

La autoridad para detener el servicio afectado sigue vigente después del cierre. También se conserva quién recibirá las señales y cómo actuará. N30 desarrollará ese seguimiento. Cerrar una etapa no extingue la responsabilidad por la operación.

El cierre necesita un equilibrio: no retirar apoyo por calendario, pero tampoco mantener un piloto especial indefinido sin decidir qué capacidad permanente hace falta. La evidencia debe permitir pasar de una excepción vigilada a una operación gobernada.

14 METSI · N29 DESARROLLO EXTENSIÓN

Transferencia: una regla de liquidación bancaria

Consideremos un cambio bancario como ejercicio conceptual de sistemas, no como procedimiento financiero listo para ejecutar. Una regla de liquidación relaciona aplicación, registros, mensajería y participantes externos. Su tratamiento real requiere controles y autoridades especializados.

Si ya se emitió una transferencia, instalar la aplicación anterior no deshace ese efecto. El plan debe distinguir operaciones pendientes, confirmadas y de estado desconocido, y reconciliarlas con las partes pertinentes. Borrar registros para que los saldos parezcan anteriores destruiría información necesaria.

Una exposición limitada puede reducir alcance, pero sólo si la frontera es real y adecuada. Un monto pequeño no vuelve aceptable cualquier error ni sustituye obligaciones. La selección requiere condiciones específicas y no se copia del ejemplo hotelero.

Ante una diferencia, la organización puede limitar nuevos envíos afectados mientras investiga los realizados. Una corrección necesita evitar duplicaciones, mantener evidencia y respetar las autorizaciones aplicables. La comunicación a personas y áreas involucradas forma parte del tratamiento de consecuencias.

La lección que se transfiere es la distinción entre componente y efecto. Podemos recuperar software y seguir teniendo una obligación pendiente o un estado externo distinto. Antes de liberar necesitamos saber qué evidencia permitirá reconocer y tratar esas diferencias.

No se exige que el estudiante diseñe aquí un protocolo de liquidación. Se le pide reconocer qué preguntas faltan cuando alguien promete «si falla, volvemos la versión». Esa pregunta bien formulada puede evitar una decisión técnicamente sencilla y operativamente riesgosa.

Contraejemplo: automatizar todos los pasos equivocados

Una organización logra que cada cambio se construya, pruebe y despliegue sin intervención manual. El tablero informa éxito. Sin embargo, las pruebas usan datos viejos y ningún paso controla una modificación directa de configuración realizada el día anterior.

La automatización funcionó como estaba diseñada. Lo que no funcionó fue la suposición de que representaba todo el sistema. Al liberar, una regla nueva se combinó con una configuración no prevista.

Agregar una firma al final tampoco resolvería por sí solo esa falta. Si quien firma sólo ve el mismo tablero, seguirá sin conocer la diferencia. Hace falta detectar configuración real, conectarla con el alcance y comprobar la combinación relevante.

El contraejemplo no niega el valor de automatizar. Muestra dónde termina su evidencia. Un proceso automático bien observado puede ser más consistente que uno manual. Pero su consistencia no compensa una pregunta ausente.

Errores frecuentes

Confundir instalación con autorización de uso. Desplegar y liberar pueden separarse, pero el despliegue también puede tener efectos de fondo que deben controlarse.

Creer que CI/CD significa siempre lo mismo. Aclarar entrega continua y despliegue continuo evita promesas contradictorias sobre cuándo un cambio llega a producción.

Equiparar hash con confianza total. Identificar contenido no demuestra autor confiable, corrección ni adecuación al servicio.

Olvidar configuración y datos. La misma aplicación puede comportarse distinto con otra regla, permiso o esquema. Registrá la combinación relevante.

Versionar secretos para dejar evidencia. Conservá referencias, responsables y proceso de provisión; no expongas credenciales en el repositorio ni en el informe.

Aprobar por cantidad de firmas. La revisión necesita información y capacidad de objetar. Repetir una aprobación sin contexto no crea independencia.

Retirar controles porque nunca fallaron. Su ausencia de bloqueos no prueba inutilidad. Revisá qué riesgo cubren y cómo aportan prevención o evidencia.

Suponer que el canario está aislado. Una base o dependencia compartida puede extender efectos fuera de la cohorte. Comprobá la frontera real.

Prometer rollback de todo porque existe un binario anterior. Verificá compatibilidad y efectos externos. Puede corresponder una corrección y reparación, no una vuelta completa.

Restaurar una copia sin reconciliar lo posterior. Las operaciones legítimas realizadas desde el respaldo no desaparecen de la responsabilidad del servicio.

Cerrar al llegar al contador del piloto. Cincuenta transiciones organizan una revisión; no demuestran todas las condiciones de calidad.

Dejar la emergencia fuera de la historia. Integrá la corrección, verificá el estado y cerrá permisos extraordinarios. Si no, la siguiente versión puede desconocer lo que funciona realmente.

17 METSI · N29 DESARROLLO EXTENSIÓN

Consecuencias profesionales

La entrega deja de medirse sólo por instalar versiones. El equipo debe mostrar una relación entre cambio, evidencia y resultado. Eso hace posible discutir qué riesgo se aceptó y qué control faltó sin reconstruir todo desde recuerdos.

También cambia el momento de involucrar a Operaciones. No se espera al final para enseñar botones. Quienes sostienen el servicio participan en escenarios, contingencias y criterios de cierre. Su conocimiento puede modificar el diseño antes de que el cambio llegue a una persona.

La aprobación se vuelve más precisa. Cambios de bajo riesgo pueden avanzar con controles estables y autoridad delegada. Los de mayor consecuencia requieren atención específica. La diferencia se justifica por efectos y condiciones, no por preferencias de un comité ni por el tamaño del pedido.

La recuperación se financia y ensaya como capacidad. No se deja como una frase optimista del proyecto. Puede incluir herramientas, permisos, copias verificadas, procedimientos y personas disponibles. Si eso tiene un costo, debe entrar en la decisión de cambio.

Finalmente, el aprendizaje vuelve al sistema de entrega. Un incidente puede mostrar una prueba ausente, un contrato confuso, una aprobación mal ubicada o una configuración no controlada. La respuesta no debería ser agregar automáticamente una firma más: debe corregir el mecanismo que produjo o amplificó el problema.

18 METSI · N29 DESARROLLO EXTENSIÓN

Límites y tensiones

Cambiar en pasos pequeños suele facilitar comprensión y recuperación, pero no elimina dependencias compartidas. Algunas modificaciones necesitan preparación coordinada. La solución es reconocer y tratar esa coordinación, no dividir tareas de manera artificial y llamar independiente a lo que no lo es.

La automatización puede reducir variación y concentrar poder. Una credencial de ejecución demasiado amplia o una regla incorrecta puede afectar muchos sistemas rápidamente. El diseño necesita permisos proporcionados, observación y formas de detenerse. No todo problema se resuelve agregando más velocidad.

La urgencia puede impedir obtener toda la evidencia deseada antes de actuar. Debe explicitarse qué falta, por qué la espera sería peor y qué límites compensan esa incertidumbre. No se afirma que una vía rápida conserva mágicamente la misma evidencia; tampoco se usa la urgencia como permiso para borrar trazabilidad u obligaciones.

La reversibilidad cambia con el tiempo. A medida que el servicio produce datos y compromisos, aumenta lo que no vuelve con una instalación. Por eso el plan debe revisar sus ramas después de ciertos eventos, no sólo después de una cantidad fija de minutos.

En 2026, herramientas de IA pueden proponer código, configuración o reparaciones. En esta lectura, la pregunta específica es cómo entra esa propuesta en la cadena controlada. Su origen no reemplaza revisión ni prueba, y una sugerencia de migración no debe ejecutarse sobre datos reales por parecer convincente. La autoridad y la evidencia se vinculan al efecto, no a la fluidez de la explicación generada.

El mismo cuidado vale para una reparación automática. Si no se puede reconocer el conjunto afectado o verificar el resultado, automatizar puede extender una interpretación equivocada. Primero se limita y se investiga; después se decide qué parte puede ejecutarse con seguridad.

Una lectura situada desde América Latina

Flores y Echeverría permiten distinguir cerrar una tarea de cumplir un compromiso. Federico puede haber instalado correctamente una versión y el hotel seguir debiendo una respuesta al huésped. La reparación no desaparece cuando termina el trabajo técnico.

Bunge exige explicar cómo se produjo el resultado. Un pipeline favorable no es una causa suficiente de buen servicio: depende de lo que verifica y de condiciones que pueden quedar afuera. García permite reconstruir esas relaciones sin reducirlas a una falla aislada de una persona.

Etkin y Schvarstein ayudan a observar prioridades organizacionales. Si la empresa premia únicamente cumplir fechas, quienes conocen un riesgo pueden sentirse empujados a callarlo o trasladarlo. Cambiar la herramienta de despliegue no corrige por sí solo ese incentivo.

Scolari recuerda que las interfaces forman parte de relaciones más amplias. Una nueva pantalla modifica cómo se coordinan Recepción, Housekeeping y huéspedes. La migración también es de prácticas y significados, aunque no todo ese cambio quede en código.

Varsavsky permite discutir qué capacidad técnica conviene construir y para qué necesidades. La CEPAL aporta contexto sobre capacidades desiguales: no todos los equipos cuentan con especialistas de guardia o infraestructura duplicada. Eso no justifica controles ficticios; exige diseñar un alcance que la organización realmente pueda sostener.

Maturana y Varela invitan a examinar las distinciones con las que observamos. Si llamamos «entrega terminada» al despliegue, dejaremos fuera conciliación y reparación. Cambiar esa definición modifica qué trabajo se reconoce y quién puede advertir que todavía falta algo.

20 METSI · N29 DESARROLLO EXTENSIÓN

De N29 a N30

HH-29 deja una versión identificable, un alcance autorizado y una recuperación ensayada. Todavía falta observar cómo se comporta durante el uso habitual, con turnos, demanda y dependencias que siguen variando.

N30 trabajará sobre señales, objetivos de servicio e incidentes. Necesitará distinguir salud de componentes y resultado para las personas. La historia del cambio permitirá investigar sin atribuir automáticamente toda dificultad al último despliegue.

21 METSI · N29 PREPARACIÓN NÚCLEO

Síntesis

Integrar, desplegar y liberar son pasos diferentes. Una cadena automatizada puede ejecutarlos de manera confiable, pero sólo demuestra las condiciones que realmente verifica. Código, configuración, datos y operación deben relacionarse para comprender el cambio.

La recuperación también tiene partes. Volver una aplicación puede ser correcto y resultar insuficiente. Corregir datos, reconciliar mensajes y atender consecuencias externas requieren otras acciones. El plan debe reconocer cuándo corresponde cada una.

HH-29 convierte esas distinciones en una práctica: identificar, probar, autorizar un alcance, observar y responder. La velocidad tiene valor cuando reduce incertidumbre y facilita reparar; no cuando vuelve más rápida una exposición que nadie puede explicar.

La imagen reúne la tensión central de la lectura y conserva abierta la decisión que sigue.

Cinco píldoras para recordar

01

Instalar una versión no demuestra que el servicio esté listo para usarla.

02

CI/CD necesita aclarar qué se automatiza y dónde se autoriza la exposición.

03

El mismo código puede producir otro resultado con distintos datos, permisos o configuración.

04

Volver el programa no deshace todos los efectos que ya produjo.

05

Una liberación termina cuando la operación puede sostenerla, no cuando el tablero se pone verde.

Del concepto al producto

RECORRIDO DE SÍNTESIS · N29

1 Identificar la combinación

El comportamiento depende de artefacto, configuración, datos, contratos y operación. La misma aplicación puede producir otro resultado.

↓

2 Autorizar el alcance

Desplegar y liberar son decisiones diferentes. La cohorte necesita una frontera real y señales de detención.

↓

3 Elegir la recuperación

Comprobar compatibilidad y efectos externos antes de volver. Si no alcanza, corregir hacia adelante y reconciliar.

↓

4 Cerrar con evidencia

Recepción debe sostener el servicio sin apoyos ocultos. Recuperar código no cierra reparaciones a personas.

PRODUCTO PARA EL ENCUENTRO

Una ficha de cambio que separe artefacto, datos y efectos sobre personas, con dos ramas de recuperación y una comprobación de servicio recuperado en cada una.

Síntesis de esta lectura. Las relaciones remiten a los conceptos y al caso desarrollados en el texto.

Glosario esencial

Integración continua, CI: incorporación y verificación frecuente de cambios pequeños en una base compartida.

Entrega continua: capacidad de mantener cambios preparados para liberarse mediante un proceso confiable, con la decisión de exposición que corresponda.

Despliegue continuo: paso automático a producción de cambios que superan controles definidos. La autoridad queda expresada en políticas y mecanismos, no desaparece.

Pipeline: secuencia automatizada de pasos de construcción, prueba o entrega. Su resultado depende de lo que comprueba.

Git: sistema de control de versiones. Conserva historia y relaciones entre revisiones; no aprueba por sí solo su calidad.

GitHub: plataforma de colaboración que agrega revisiones, automatizaciones y gestión alrededor de repositorios.

Artefacto: unidad construida que se distribuye o instala. Debe identificarse y vincularse con fuente, dependencias y pruebas.

Configuración: parámetros y opciones que influyen en el comportamiento. Puede cambiar consecuencias sin cambiar el código principal.

Deriva: diferencia entre estado real y estado controlado esperado. Requiere investigar antes de corregir automáticamente.

Infraestructura como código: definiciones revisables y ejecutables de recursos y reglas. Tenerlas no demuestra que la recuperación esté ensayada.

Puerta de decisión: evaluación de condiciones para asumir el siguiente compromiso, con autoridad y resultados posibles.

Segregación de funciones: separación de capacidades sensibles para reducir abuso o error sin detectar.

Exposición progresiva: habilitación por alcances controlados, con señales y condiciones de ampliación o detención.

Rollback: vuelta a una versión o estado anterior dentro de un alcance. Puede ser sólo de código y no recuperar todo el servicio.

Rollforward: corrección hacia adelante desde el estado actual. Necesita alcance y verificación, no improvisación.

SBOM: lista de materiales de software. Ayuda a identificar componentes, pero no certifica su seguridad.

Procedencia: relación entre artefacto, origen y transformaciones. Debe conectarse con lo realmente desplegado.

Cierre de liberación: comprobación de que el cambio puede sostenerse en operación, con responsabilidades y pendientes tratados.

Preguntas de preparación

1. ¿Qué diferencia hay entre CI, entrega continua y despliegue continuo? Mostralo con una regla nueva del hotel.
2. ¿Cómo comprobarías que el artefacto instalado es el que se examinó? ¿Qué seguiría sin demostrar esa identidad?
3. ¿Qué modificación de configuración podría cambiar el servicio sin modificar la aplicación?
4. ¿Qué información necesitás antes de decidir un rollback de HH-29?
5. ¿Qué efectos requieren reparación aunque vuelva la versión anterior?
6. ¿Cómo comprobarías que la cohorte del piloto no afecta indirectamente a otras personas?
7. ¿Qué tendría que saber y poder hacer Recepción antes de retirar el apoyo reforzado?

Para el encuentro, prepará un plan de cambio con dos ramas de salida: reversión compatible y corrección hacia adelante. Usá doce unidades de prueba, separá estado del sistema y efecto sobre personas, fijá señal de detención, responsable y prueba de recuperación. Entregá el árbol de decisión y el registro antes/después; indicá qué resultado te haría abandonar cada rama.

ENSAYO EN PAPEL · ANTES DE ABRIR LA HERRAMIENTA

Seguí cada salida hasta una prueba de servicio. Dejá visible qué dato te haría cambiar de rama.

01 · SEÑAL DE DETENCIÓN	02 · RAMA Y ALTERNATIVA	03 · PRUEBA DE RECUPERACIÓN

Referencias base

Humble, J. y Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.

Forsgren, N., Humble, J. y Kim, G. (2018). *Accelerate*. IT Revolution.

DeBellis, D. et al. (2024). *DORA Accelerate State of DevOps 2024 Report*. https://research.google/pubs/dora-accelerate-state-of-devops-2024-report/

Souppaya, M., Scarfone, K. y Dodson, D. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST SP 800-218. https://doi.org/10.6028/NIST.SP.800-218

Boyens, J. M., Smith, A., Bartol, N., Winkler, K., Holbrook, A. y Fallon, M. (2024). *Cybersecurity Supply Chain Risk Management Practices for Systems and Organizations*. NIST SP 800-161 Rev. 1 Update 1. https://doi.org/10.6028/NIST.SP.800-161r1-upd1

Chandramouli, R., Kautz, F. y Torres-Arias, S. (2024). *Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines*. NIST SP 800-204D. https://doi.org/10.6028/NIST.SP.800-204D

SLSA (2025). *SLSA Specification 1.2*. https://slsa.dev/spec/v1.2/

CISA (2023). *Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software*. https://www.cisa.gov/resources-tools/resources/secure-by-design

ISO/IEC (2022). *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection: Information security management systems requirements*. https://www.iso.org/standard/27001

ISO/IEC (2018). *ISO/IEC 20000-1:2018 Information technology, Service management, Part 1: Service management system requirements*. https://www.iso.org/standard/70636.html

OpenGitOps (2021). *GitOps Principles 1.0*. https://opengitops.dev/

Kim, G., Humble, J., Debois, P. y Willis, J. (2021). *The DevOps Handbook*, Second Edition. IT Revolution.

Beck, K. y Andres, C. (2004). *Extreme Programming Explained*, Second Edition. Addison-Wesley.

Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2.ª ed.). Addison-Wesley.

Edmondson, A. C. (1999). "Psychological Safety and Learning Behavior in Work Teams". *Administrative Science Quarterly*, 44(2), 350 a 383. https://doi.org/10.2307/2666999

Vogels, W. (2009). "Eventually Consistent". *Communications of the ACM*, 52(1), 40 a 44. https://doi.org/10.1145/1435417.1435432

Chacon, S. y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2

GitHub (2026). *About protected branches*. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

Flores, F. (1997). *Creando organizaciones para el futuro*. Dolmen.

Bunge, M. (2004). *Emergencia y convergencia: novedad cualitativa y unidad del conocimiento*. Gedisa.

García, R. (2006). *Sistemas complejos: conceptos, método y fundamentación epistemológica de la investigación interdisciplinaria*. Gedisa.

Scolari, C. A. (2018). *Las leyes de la interfaz: diseño, ecología, evolución, tecnología*. Gedisa.

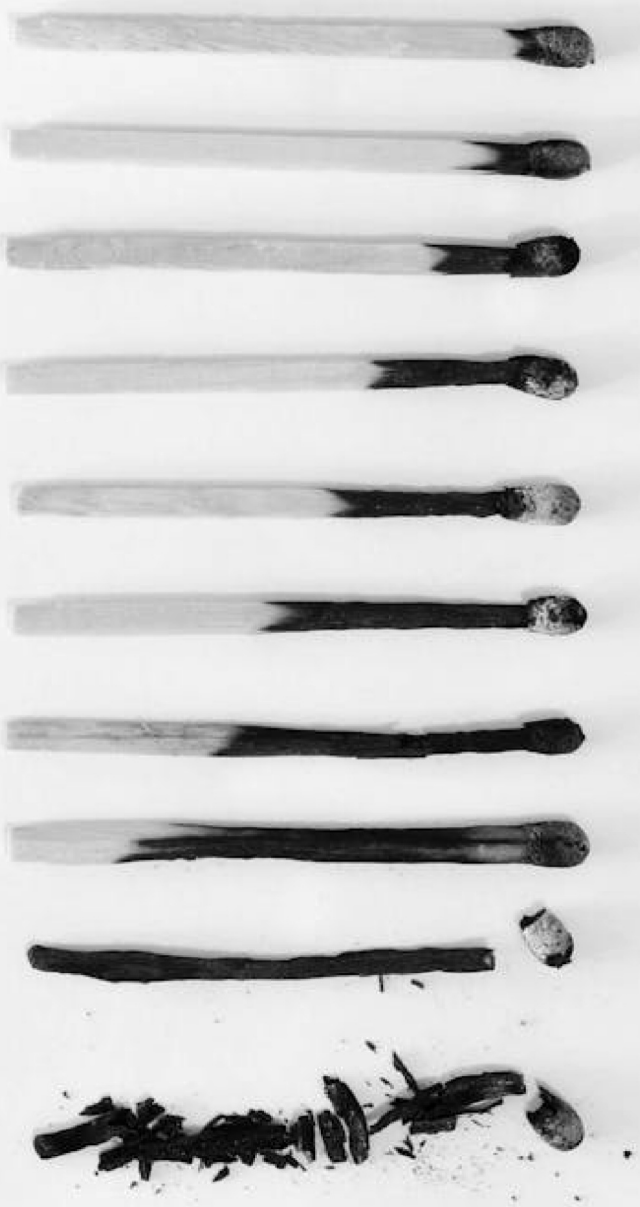
Etkin, J. y Schvarstein, L. (1989). *Identidad de las organizaciones: invariancia y cambio*. Paidós.

Maturana, H. R. y Varela, F. J. (1984). *El árbol del conocimiento: las bases biológicas del entendimiento humano*. Editorial Universitaria.

Varsavsky, O. (1972). *Hacia una política científica nacional*. Ediciones Periferia.

CEPAL (2022). *Un camino digital para el desarrollo sostenible de América Latina y el Caribe*. Naciones Unidas.

Echeverría, R. (2005). *Ontología del lenguaje*. Granica.



La secuencia vuelve visible que toda intervención consume recursos, deja huellas y necesita un criterio de cierre.